

# Pitch video script: Bob Onboarding Pipeline (5:00 version)

Audience: IBM Bob 2.0 hackathon judges (lablab.ai). Criteria to hit: Application of Technology, Presentation, Business Value, Originality. Speaking pace ~130 words/min. [X] = fill in before recording; do not guess.

**Rule check:** the actual submission requirement is  $\leq 3:00$  — see `video-script.md` for that cut. This 5:00 version is kept as a fuller pitch/reference: same content, same visuals, delivered at normal pace over 5 minutes instead of 3. If you'd rather keep this content but fit it into 3:00, speak faster and hold shots shorter rather than cutting slides — see the note at the bottom.

**Source of truth:** pulled directly from the speaker notes on the 13-slide pitch deck (Team Onboard — [deck link](#)), a separate artifact from the current 3:00 deck. Times are cumulative, word-count  $\div 130$  wpm, rounded to the nearest few seconds.

## 5:00 script

| Time      | Slide         | On screen                                                                                                | Narration                                                                                                                                                                                                                                                                                                                                                                                                            |
|-----------|---------------|----------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 0:00–0:23 | cover         | Title card: "Day one, a stale README." Slow / Expensive / Inconsistent                                   | "A new developer joins. Day one, they get a wiki link and a stale README. Their senior spends days explaining the same things: where the code starts, what's broken, which tools to install. That's the onboarding problem: slow for the newcomer, expensive for the team, and different every time."                                                                                                                |
| 0:23–0:47 | problem-cost  | Two stats: 58% (Xia et al.), 82% (LaToza et al.)                                                         | "This isn't just an anecdote. A field study of seventy-nine professional developers found they spend fifty-eight percent of their time simply understanding code, not writing it. And eighty-two percent say the hardest part is knowing why the code is the way it is, knowledge that usually lives only in someone's head."                                                                                        |
| 0:47–1:13 | problem-fixes | Diagram: new hire → wiki / senior dev / trial-and-error, all dead ends. Callout: LACY 57% vs 83%         | "The usual fixes don't scale. Wikis go stale. Senior developers get pulled into the same explanation for every new hire. And going AI-only doesn't solve it either — one industry deployment found self-guided AI tours scored fifty-seven percent on comprehension, against eighty-three percent when a human curates first. That's exactly why our pipeline has a human gate."                                     |
| 1:13–1:42 | solution      | Three cards: Personalized / Grounded in code / Never self-graded, plus the <code>/onboard</code> command | "We built an onboarding pipeline inside Bob IDE. A manager types one command, <code>/onboard</code> . Bob interviews them, scans the real codebase, and produces a personalised pack for that hire. The design follows the onboarding research: personalise by role and experience, ground every claim in the code, and deliver through more than one channel. And crucially, it never lets the model grade itself." |
| 1:42–2:04 | pipeline      | Flowchart: Intake → Generate $\times 4$ → Verify → Approve → Render → Send                               | "Here's the shape. A Bob skill drives the stages. Four subagents run in parallel: company and role, architecture and code tour, an issue scan, and tech stack. Then plain Python checks every citation against the repo. Then a human gate. Only after approval does it render and send."                                                                                                                            |
| 2:04–2:20 | live-intake   | <b>LIVE</b> screenshot: Bob asking for the target codebase path                                          | "Live run. The manager points Bob at a codebase and confirms the project. The client's company, brand and repo come from an instance file, so the core pipeline stays generic. Intake takes about two minutes."                                                                                                                                                                                                      |
| 2:20–2:52 | live-generate | <b>LIVE</b> screenshot: four subagents launching in parallel, "10 issues found" badge                    | "Four subagents work at once. Bob finds ten real issues in this Java app, including three sets of hardcoded credentials, and writes one file per issue with the evidence at file and line, plus guidance, not a patch. Then the deterministic check runs. It verifies every path and line exists. Anything it can't confirm goes into a flagged list, marked 'Unknown, ask your buddy'. Bob never invents a fact."   |

| Time      | Slide         | On screen                                                                                               | Narration                                                                                                                                                                                                                                                                                                                                                                                                                        |
|-----------|---------------|---------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2:52–3:05 | live-approval | LIVE screenshot: the approval gate, flagged issues + decision options                                   | "Now the human gate. The manager sees only what needs judgement: the flagged items and the issue index. Tool auto-approve doesn't count. A person has to say yes."                                                                                                                                                                                                                                                               |
| 3:05–3:24 | live-outputs  | LIVE screenshot: rendered PDF, table of contents and welcome cards                                      | "Approved. Bob renders a branded PDF, a narrated code-tour video, and a context folder. That folder is the interesting part: the newcomer loads it into their own Bob, so they get an assistant that already knows their company, role, and codebase."                                                                                                                                                                           |
| 3:24–3:39 | live-send     | LIVE screenshot: Ethereum inbox, email received with 3 attachments                                      | "Delivery goes through an email MCP server we had Bob build. It's stdlib only, SMTP from environment variables, with a dry-run mode. Newcomer gets the pack; the manager gets a separate brief."                                                                                                                                                                                                                                 |
| 3:39–4:10 | bob-usage     | Four cards: skill+command, parallel Agent mode, Bob built its own MCP server, sessions logged           | "A quick note on how we used Bob. It's a skill plus a command, not a one-off script: <code>/onboard</code> and <code>/build-pipeline</code> . Parallel subagents handle the four generate tasks, in Agent mode end to end. Bob built the email MCP server itself and sent the newcomer email through it, and it ran its own fix loop when a stage failed. Every task's summary is screenshotted in <code>bob_sessions/</code> ." |
| 4:10–4:37 | impact        | Big numbers: ~20 min vs. 20–40 hrs (LACY, Beko)                                                         | "Impact. The run took about 20 minutes of machine time plus one minute of human approval. Industry data on manual onboarding — expert time spent per new hire, mentoring and repeating the same explanations — runs 20 to 40 hours, per the LACY deployment study at Beko. The narrated video is the slowest stage in our run, and it's optional."                                                                               |
| 4:37–4:57 | close         | Closing statement + <code>/build-pipeline</code> · <code>instances/&lt;client&gt;</code> · Team Onboard | "It's a proof of concept: one demo project so far. But the core is generic. A new client is a folder, and Bob can rebuild the whole pipeline from its spec. Faster ramp-up, grounded in real code, with a human in control. Thanks."                                                                                                                                                                                             |

Runs ~4:57 read at a steady 130 wpm — that's narration-only; live stage transitions and letting judges actually read the screenshots (especially the issues table and the PDF) will likely push the real run to 5:30–6:00.

## If you want this content but need to land at 3:00

Two options, in order of preference:

1. **Use the actual 3:00 cut instead** ( `video-script.md` , deck at [the 3:00 artifact](#) ) — it already compresses the problem section to one line, merges `pipeline` into `solution` , and drops `bob-usage` from the main flow. This is what should go to lablab.
2. **Compress this 5:00 script's delivery to 3:00** — speak at ~215 wpm instead of 130 (fast but doable for a rehearsed script), and hold each live screenshot for roughly 60% of the time listed above. This keeps every slide and stat but is a harder recording to nail in one take than option 1. Only do this if you specifically need all 13 slides on screen (e.g., a judge Q&A follow-up) rather than for the actual submission video.

## Recording tips

- Record the live run once, then cut the waiting; narrate over the edit rather than live. Show real terminal and Bob UI, since judges want proof it runs, not a mock-up.
- Video must show Bob's role in every stage; keep the Bob IDE panel visible.
- The `bob-usage` beat (3:39–4:10) speaks directly to the *Application of Technology* judging criterion.